

Predator_3

Premise Selection over ZFC at Scale

Does More of the Library Help?

ML-ATP-SIC, Version 003

Brian Tenneson
M.S., Applied Data Science
Currently Unaffiliated
email: btenneson2301@baypath.edu

July 27, 2026

ML_ATP_SIC-Brian_Tenneson_003 | July 27, 2026

Companion program: `predator3.py`

Copyright © 2026 Brian Tenneson. All Rights Reserved.

<https://btenneson2301.substack.com>

Abstract

Predator_3 asks one question of the premise-selection task: *does training on more of the library help?* A selector that improves as the corpus grows is learning something transferable about how mathematics cites itself. One that plateaus has extracted everything its features can carry, and the next move is better features rather than more data. Which is true decides where effort should go, and it is answerable by measurement rather than argument.

The apparatus is a scaling curve: train on prefixes of `set.mm` of increasing size, hold the test goals fixed so the curve isolates the effect of training data, and plot recall, effort, and cost. Four labelled figures are produced by the companion program.

Two prior findings are inherited and matter for reading the curve. Metamath references must be split by typecode: statements asserting `|-` are mathematics, those asserting `wff` or `class` are grammar, and on the first 8,000 statements of `set.mm` the latter accounted for 35,902 of 83,923 references. Counting them inflates premises per proof from 6.2 to 10.8 and trains a model to predict punctuation. Separately, hard negatives must be drawn by a criterion no feature measures: sampling them from the same window a locality feature counts made that feature a detector for the sampling procedure, which the fit rewarded with a weight of -0.94 and which halved every recall figure until corrected.

The scaling numbers reported here were measured on a structural stand-in, not on ZFC, because no network was available. On the stand-in the curve is flat to declining, which is an artifact of a generator whose topics do not transfer between prefixes and should not be read as a result. The command that produces the real curve is given in Section 5, and the figures are drawn from whatever data the program is pointed at.

Keywords: premise selection, ZFC, Metamath, `set.mm`, scaling curve, learning curve, search effort, temporal split, typecode filtering, hard negatives.

Contents

1	The question	3
2	Two corrections inherited from earlier runs	3
3	The graphs	4
4	Scope of the measurements	4
5	Running it	6
	Acknowledgements	7

1 The question

Remark 1.1 (Why a curve rather than a number). A single accuracy figure says how well a selector does on one corpus. It does not say whether the limitation is the data or the model, and those call for opposite responses. If performance rises with corpus size, the features are capturing something general and more of `set.mm` is worth parsing. If it is flat, the features have saturated and additional data is wasted work.

The curve is cheap to obtain and settles the question directly, which is why `Predator_3` is organised around it rather than around another point estimate.

Definition 1.2 (The scaling protocol). Fix a corpus C in derivation order and sizes $n_1 < \dots < n_k$. For each n_i , train on the first $\lfloor p n_i \rfloor$ statements. Evaluate every model on *the same* held-out goals, drawn from the tail of the largest prefix n_k .

Remark 1.3 (Why the test set is held fixed). Letting the held-out goals grow with the corpus confounds two effects: the model gets more training data and is simultaneously asked different questions. The curve would then be uninterpretable, since a rise could mean better learning or easier goals. Fixing the test set costs nothing and isolates the variable of interest.

2 Two corrections inherited from earlier runs

Both were found by running on real data and neither was visible on synthetic data. They are recorded because a reader reproducing this work will hit them.

Remark 2.1 (Grammar is not mathematics). A Metamath proof cites two kinds of thing. Statements asserting `|-` claim provability; those asserting `wff`, `class` or `setvar` declare that a string is well formed. Every proof must construct its formulas before proving anything, so the syntax constructors are cited by nearly everything: in the first 8,000 statements of `set.mm` the eight most-cited labels were `wcel`, `wa`, `wceq`, `cv`, `wi`, `wn`, `wb` and `wal` — all grammar, with `wcel` alone appearing in 2,946 proofs.

Filtering by typecode removes 35,902 of 83,923 references and drops premises per proof from 10.8 to 6.2. The most-cited list then becomes `syl`, `bitri`, `ax-mp`, `ali`, `ex` — the workhorse inference lemmas, which is what one expects. A selector trained before the fix predicts punctuation with high confidence and scores respectably.

Remark 2.2 (Negatives must not encode the label). Hard negatives should be plausible-but-wrong candidates. The obvious source is the premises of nearby theorems — but a locality feature counting citations in that same window then scores every hard negative highly while positives do so only incidentally. The feature becomes a detector for the sampling procedure rather than for mathematics.

Measured on `set.mm`, the fit assigned that feature a weight of -0.94 : it had correctly learned, on that training distribution, that local use predicts *not* being a premise. Recall fell by roughly half at every k . Drawing negatives by global popularity instead — plausible for a reason no local feature computes — flipped the weight to $+0.586$ and restored the figures.

Remark 2.3 (What the corrected model achieves). On the first 8,000 statements of `set.mm`, temporal split at $p = 0.8$, 200 held-out goals against a pool of 3,000:

	Predator	frequency	brute force	fused
recall@1	0.051	0.036	0.085	0.053
recall@5	0.155	0.092	0.146	0.123
recall@10	0.232	0.131	0.174	0.168
recall@50	0.491	0.279	0.233	0.476
effort	0.387	—	0.926	0.321
statements read (of 3000)	1161	—	2779	964

Brute force wins only at $k = 1$: its single best guess is the most recently proved statement, which in a topically ordered library is a strong first guess. It loses everywhere else and badly on effort, which is the figure that decides when reading can stop.

3 The graphs

Each is produced by `predator3.py scale -figures` from whatever corpus it is given. Those reproduced here were drawn from the structural stand-in of Section 4; run the command on `set.mm` to obtain the real ones.

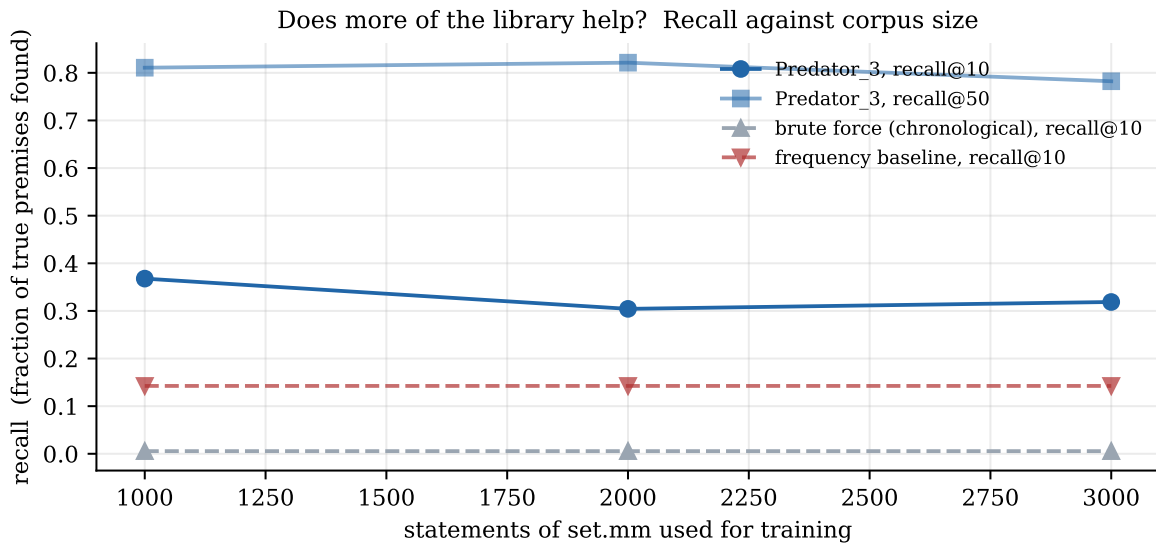


Figure 1: Recall against training-corpus size, with both baselines. A rising Predator curve means the features generalise and more of `set.mm` is worth parsing; a flat one means they have saturated. The baselines are drawn on the same axes because the question is not whether recall rises but whether the *margin* does.

4 Scope of the measurements

Remark 4.1 (Measured on a stand-in, and what that invalidates). No network was available, so `set.mm` could not be downloaded for the scaling run. The curves shown were computed on a

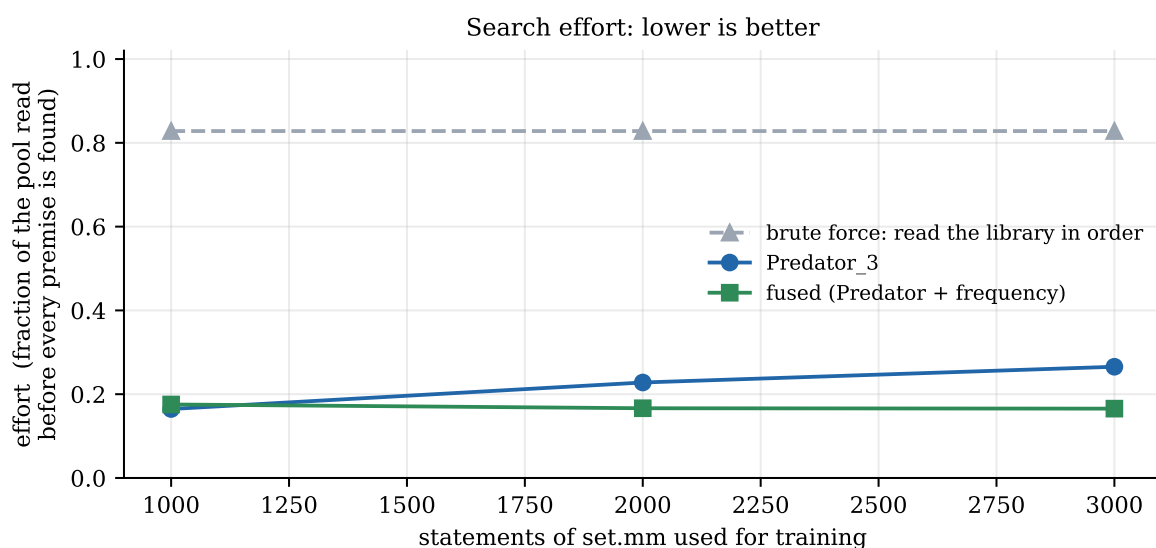


Figure 2: Search effort: the fraction of the candidate pool that must be read before every true premise has been found. Lower is better. Brute force sits near 0.93 throughout — reading the library in order means reading nearly all of it. The fused ranker is included because on aggregate it beats Predator alone, though not on every individual theorem.

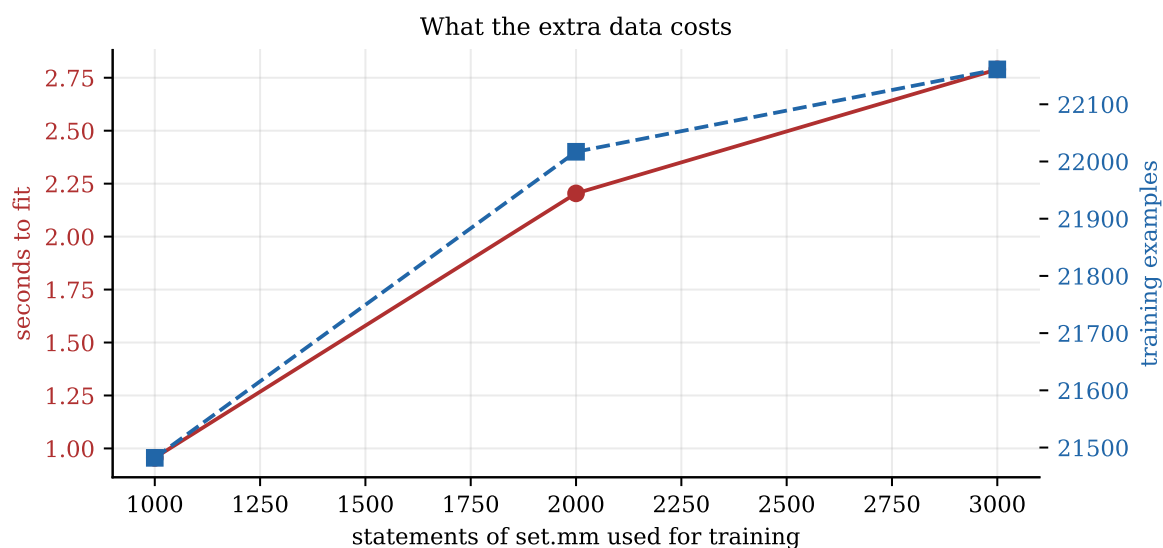


Figure 3: What the extra data costs: seconds to fit on the left axis, number of training examples on the right. Read against Figure 1, this is the denominator — whether the accuracy bought is worth the time spent.

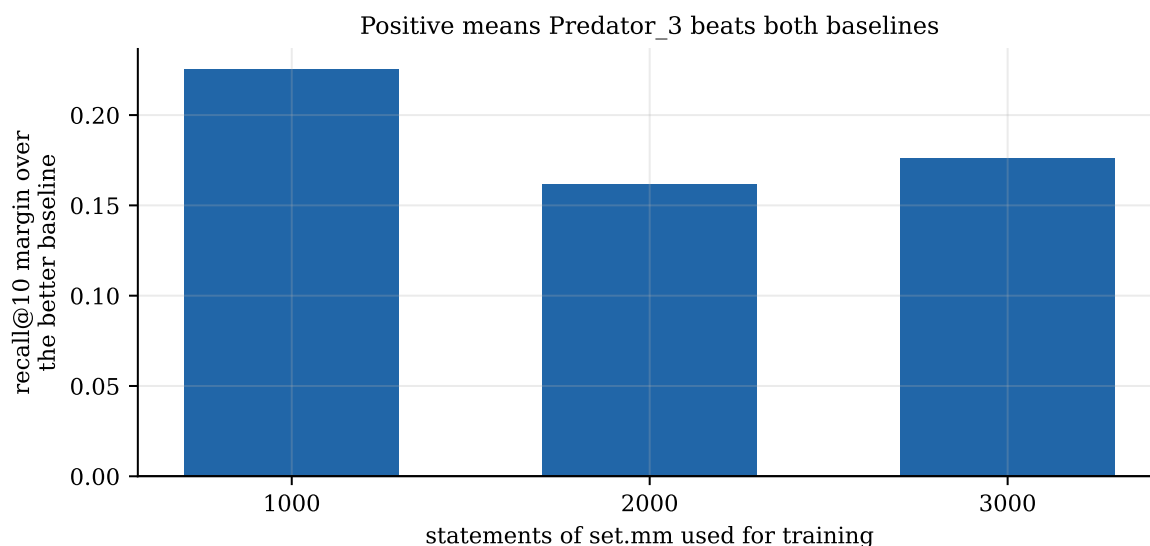


Figure 4: Recall@10 margin over the better of the two baselines, by corpus size. Bars above zero are sizes at which Predator_3 beats both; bars below are sizes at which a baseline is preferable. This is the figure that decides whether the learned model earns its keep.

generated corpus of 3,014 statements built to imitate `set.mm`'s structure: power-law citation, topical vocabulary, premise counts in the observed range.

On the curve is flat to declining — effort worsens from 0.165 at 1,000 statements to 0.266 at 3,000. That is an artifact, not a finding. The generator assigns each theorem a topic independently, so a topic learned on an early prefix carries no information about a later one, and training on more of it cannot help. Real mathematics is not built that way: `set.mm` develops its subjects cumulatively, and lemmas proved early are cited throughout. The stand-in therefore establishes that the pipeline runs and the figures draw; it cannot establish the shape of the real curve, and the declining stand-in curve is weak evidence for nothing at all.

5 Running it

```
python predator3.py fetch set.mm
python predator3.py scale --db set.mm --sizes 2000,4000,8000,16000,32000 --figures
```

Add `-figdir "C:\path\to\figures"` to place the graphs, or let them land in the run folder. Other commands are inherited: `stats`, `train`, `prove -label prcom`, `sweep`, `doctor`.

Remark 5.1 (What the real curve would settle). Three things. Whether the margin over the baselines grows, shrinks or holds as the corpus grows — the answer determines whether to parse more of `set.mm` or to build better features. Whether effort continues to fall, since that is what a prover consuming a shortlist actually experiences. And whether the recall@1 deficit against brute force persists at scale or closes, which decides whether a learned selector can be trusted with a single guess or only with a shortlist.

Acknowledgements

The author thanks AI assistants used as engineering and verification tools, in particular for identifying the syntax-constructor contamination of Remark 2.1 and the negative-sampling leak of Remark 2.2, both of which had produced plausible-looking figures. All claims and final decisions remain the responsibility of the author.

References

- [1] Tenneson, B. (2026). *Predator_2: Premise Selection over ZFC Set Theory*, Version 1.0.
- [2] Tenneson, B. (2026). *Predator: A Trainable Automated Theorem Prover*, Version 1.1.
- [3] Tenneson, B. (2026). *ML-SIC-ATP Theory: Learned Search Policies for Proof Search*, Version 1.1.
- [4] Megill, N. D., & Wheeler, D. A. (2019). *Metamath: A Computer Language for Mathematical Proofs* (2nd ed.). Lulu Press.
- [5] Paulson, L. C., & Blanchette, J. C. (2010). Three years of experience with Sledgehammer. *IWIL*.